



Programación de Estructuras de Datos y Algoritmos Fundamentales

Actividad Integradora 4 Grafos y Algoritmos

Farid Gabriel Velasco Martínez A01736669

26 de mayo de 2025

Índice

Índice	2
Introducción	3
Binary Heap como estructura auxiliar y complementaria en el grafo	3
Análisis de eficiencia y complejidad computacional	4
Evaluación del camino de mayor esfuerzo	4
Pseudocódigo del método empleado para identificar el camino entre el bot master y la dirección IP más costosa	5
Conclusión	6
Referencias:	7

Introducción

En esta cuarta actividad integradora se aborda el análisis computacional de registros de conexión entre direcciones IP, considerando tanto la temporalidad como la intensidad (peso) de las mismas. A partir de esta información, se construye un modelo en forma de grafo dirigido ponderado, una estructura de datos fundamental para representar relaciones complejas entre entidades. En este contexto, cada nodo representa una dirección IP, y cada arista una conexión con su respectivo peso, permitiendo modelar y analizar la propagación de conexiones, identificar patrones y detectar actores relevantes dentro de la red, como el bot master.

Aunque otras estructuras de datos como arreglos o tablas hash podrían ser utilizadas para procesar ciertos aspectos de los registros, como ordenamientos o conteos, los grafos proporcionan una representación más adecuada y flexible del sistema en su conjunto. Además, permiten aplicar algoritmos de análisis topológico y de caminos como el de Dijkstra, optimizando la extracción de información clave sobre rutas y grados de conexión.

Binary Heap como estructura auxiliar y complementaria en el grafo

El Binary Heap cumple un papel central como cola de prioridad en la implementación del algoritmo de Dijkstra. Su función es mantener ordenados los nodos por distancia mínima desde el nodo origen, permitiendo realizar extracciones eficientes en tiempo logarítmico. Esta eficiencia ($\theta(\log V)$) contrasta favorablemente con estructuras más simples como listas, que operarían en tiempo lineal ($\theta(V)$) al buscar el siguiente nodo más cercano.

En esta actividad se desarrolló la clase `ALDWGraph`, la cual modela el grafo mediante una lista de adyacencia optimizada para inserciones, consultas y recorridos. Su integración con la estructura Binary Heap permitió resolver de forma eficiente los problemas de caminos más cortos, detección de nodos con mayor grado, y determinación del camino de mayor esfuerzo entre IPs.

Análisis de eficiencia y complejidad computacional

La elección de grafos como estructura base fue adecuada por su capacidad para representar relaciones dirigidas y ponderadas. Su complejidad espacial $\theta(V + E)$ y su compatibilidad con algoritmos clásicos hacen que sean ideales para problemas de conectividad y propagación en redes.

Respecto al algoritmo de Dijkstra, su implementación con un Binary Heap garantiza una complejidad temporal de $\theta((V + E) \log V)$, lo cual lo hace escalable para grafos de tamaño considerable. Si se hubieran utilizado otras estructuras menos eficientes, como listas o sets ordenados, el tiempo de ejecución podría verse afectado negativamente, especialmente ante grandes volúmenes de datos.

En conjunto, la combinación de estructuras como `unordered_map`, `vector`, `list` y `Heap` ofrece un balance óptimo entre eficiencia y flexibilidad, permitiendo que el sistema procese incluso archivos con miles de entradas sin degradar su rendimiento.

Evaluación del camino de mayor esfuerzo

El camino de mayor esfuerzo representa la ruta más costosa, desde el bot master hacia otra IP, en términos de suma de pesos. Esta métrica es fundamental para la ciberseguridad, ya que permite identificar rutas críticas o vulnerables, que podrían requerir mayor monitoreo o refuerzo.

El análisis se realizó extrayendo el conjunto de caminos generados por Dijkstra y seleccionando aquel con mayor distancia acumulada. Esta operación no sólo cuantifica la dificultad de acceso, sino que también puede correlacionarse con el nivel de aislamiento o centralidad de una IP en la red.

Pseudocódigo del método empleado para identificar el camino entre el bot master y la dirección IP más costosa

Complejidad: $O(V + E \log V)$

Entrada: nodo de inicio 'start'

Salida: lista de estructuras Dijkstra_path_info con caminos más cortos desde 'start'

1. Si 'start' no existe en el grafo, retornar lista vacía.

2. Inicializar:

- mapa 'distances' con todas las distancias en infinito
- mapa 'parent' con todos los valores en nullptr
- mapa 'visited' en falso
- heap de prioridad 'pq' que ordena por menor distancia

3. Establecer $\text{distance}[\text{start}] = 0$

Insertar (0, start) en 'pq'

4. Mientras 'pq' no esté vacío:

4.1 Extraer nodo 'node' con menor distancia 'dist' de 'pq'

4.2 Si 'node' ya fue visitado, continuar

4.3 Marcar 'node' como visitado

4.4 Para cada vecino 'neighbor' de 'node':

- Calcular distancia nueva: $\text{new_dist} = \text{dist} + \text{peso_arista}$

- Si $\text{new_dist} < \text{distance}[\text{neighbor}]$:
 - Actualizar $\text{distance}[\text{neighbor}] = \text{new_dist}$
 - Actualizar $\text{parent}[\text{neighbor}] = \text{node}$
 - Insertar $(\text{new_dist}, \text{neighbor})$ en 'pq'

5. Para cada nodo alcanzado distinto de 'start':

5.1 Si $\text{distance}[\text{node}]$ sigue siendo infinito, continuar

5.2 Reconstruir camino desde 'start' a 'node' usando 'parent'

5.3 Almacenar camino y distancia en lista 'result'

6. Retornar 'result'

Conclusión

Esta actividad permitió consolidar conocimientos sobre estructuras de datos avanzadas y su aplicación a problemas reales de análisis de redes. La correcta elección de estructuras como grafos y heaps, así como la modelación adecuada del problema, fueron clave para alcanzar soluciones eficientes y robustas. El enfoque adoptado no solo resolvió los requerimientos funcionales, sino que también brindó una base para la toma de decisiones en contextos de seguridad informática.

Referencias:

1. GeeksforGeeks. (s.f.). Dijkstra's Algorithm.
<https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-graph-data-structure/>
2. cppreference.com. `std::priority_queue`, `std::unordered_map`, `std::vector`
3. *Adjacency List (With Code in C, C++, Java and Python)*. (s. f.).
<https://www.programiz.com/dsa/graph-adjacency-list>
4. GeeksforGeeks. (2024, 5 octubre). *Graph and its representations*. GeeksforGeeks.
<https://www.geeksforgeeks.org/graph-and-its-representations/>
5. GeeksforGeeks. (2025a, abril 23). *Dijkstra's Shortest Path Algorithm using priority_queue of STL*. GeeksforGeeks.
https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-using-priority_queue-stl/
6. GeeksforGeeks. (2025b, abril 23). *Dijkstra's Shortest Path Algorithm using priority_queue of STL*. GeeksforGeeks.
https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-using-priority_queue-stl/
7. Jacob Sorber. (2021, 21 septiembre). *What is a Graph Data Structure? When to use it? How to easily visualize it?* [Video]. YouTube.
<https://www.youtube.com/watch?v=tq3zPnrQIpU>